



COMP 364/464: High-Performance Computing Week 06, Lecture 06



Agenda

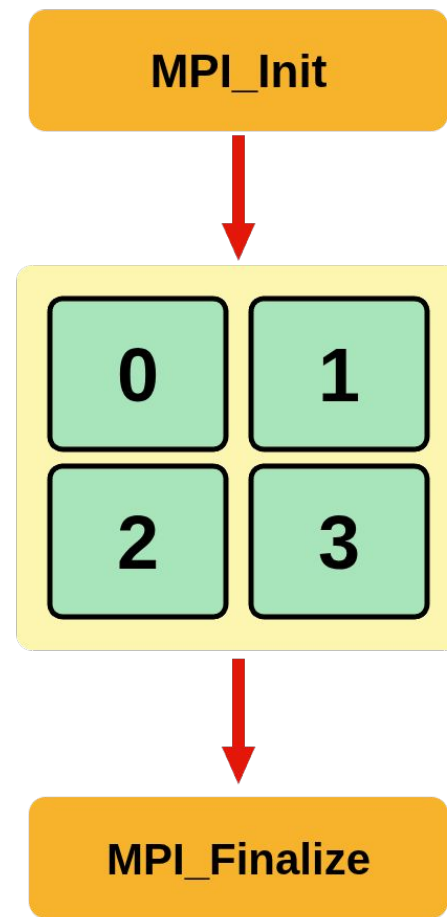
- MPI
- Practice!
- Project 01
- Project 02

What is MPI?

- SIMD system
- Distributed memory parallelism
 - Each processor has separate memory
- Specification written by MPI Forum
 - Implementations include Open MPI, MPICH, MVAPICH, MPI Intel, Cray MPICH, etc.
- Library for C, C++, and Fortran (notice a pattern?)
- Bindings exist for Python and many others
- Underlying tech behind most distributed communications platforms

How it Works

- Launch N copies of the same executable
- Each copy gets its own *rank*
- MPI subsystem gives work to each rank and offers coordination schemes



Hello, MPI

MPI_Init:

Starts the runtime

MPI_Comm_rank:

"Which process am I?"

MPI_Comm_size:

"How many processes are there?"

MPI_Finalize:

Shuts down the runtime

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    printf("Hello from rank
           %d of %d\n", rank, size);
    MPI_Finalize();
    return 0;
}
```

Compilation and Runtime

- **mpicc:**
 - Compiler wrapper
 - Wraps your C compiler and adds the MPI *include* and *link* flags
- **mpirun or mpiexec:**
 - Launches the processes
- Just like OpenMP, keep $N \text{ ranks} \leq \# \text{ processors}$

Communication

- “Communicators”:
 - Group of processes with a private communication “context”
 - **Ranks are relative to a communicator**: one process can be rank 3 in one and rank 0 in another
- MPI_COMM_WORLD contains every process that was launched
- New communicators come from MPI_Comm_split and MPI_Comm_dup
- A message sent on one communicator can never be received on another

Send and Receive

buf: Address of the first element

count: Number of elements

datatype: the type of each element

dest / source: the other process's rank

tag: a nonnegative int to label the message

comm: the communicator

status (receive only): filled in with extra information. Use MPI_STATUS_IGNORE if you don't need it

```
int MPI_Send(const void *buf, int count,
             MPI_Datatype type, int dest, int tag,
             MPI_Comm comm);
```

```
int MPI_Recv(void *buf, int count, MPI_Datatype
             type, int source, int tag, MPI_Comm comm,
             MPI_Status *status);
```

Built-In Datatypes

- MPI_CHAR: char used as text
- MPI_SIGNED_CHAR, MPI_UNSIGNED_CHAR: char used as a small integer
- MPI_SHORT, MPI_INT, MPI_LONG, MPI_LONG_LONG: short, int, long, long long
- MPI_UNSIGNED_SHORT, MPI_UNSIGNED, MPI_UNSIGNED_LONG, MPI_UNSIGNED_LONG_LONG: unsigned int, long, and long-long
- MPI_FLOAT, MPI_DOUBLE, MPI_LONG_DOUBLE: float, double, long double
- MPI_INT32_T, MPI_INT64_T, MPI_UINT8_T, etc: fixed-width types from stdint.h
- MPI_C_BOOL: _Bool
- MPI_BYTE: raw bytes

Example: Send and Receive

buf: Address of the first element

count: Number of elements

datatype: the type of each element

dest / source: the other process's rank

tag: a nonnegative int to label the message

comm: the communicator

status (receive only): filled in with extra information. Use `MPI_STATUS_IGNORE` if you don't need it

```
// Send 10 doubles starting at a
// to rank 2, labeled 5
```

```
MPI_Send(a, 10, MPI_DOUBLE,
         2, 5, MPI_COMM_WORLD);
```

```
// Receive up to 10 doubles
// from rank 0, labeled 5
```

```
MPI_Recv(b, 10, MPI_DOUBLE,
        0, 5, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Example: Ping-Pong

1. Rank 0 sends a value, then waits for the reply
2. Rank 1 receives it, changes it, and sends it back
3. Note: Must be run with at least 2 processes
4. Put it in a loop with MPI_Wtime() and it becomes a latency benchmark

```
int n = 0;
if (rank == 0) {
    n = 42;
    MPI_Send(&n, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
    MPI_Recv(&n, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("rank 0 got back %d\n", n);
}
else if (rank == 1) {
    MPI_Recv(&n, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    n += 1;
    MPI_Send(&n, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
}
```

Cautions about Deadlock

- MPI_Send returns when the send buffer is safe to reuse (not on delivery)
 - Small messages are usually buffered by the library (eager protocol), so the send returns right away
 - Large messages usually wait for the matching receive (rendezvous protocol)
- MPI_Recv holds until the copy is complete, so it too can hang

```
MPI_Send(out, N, MPI_DOUBLE, other, 0, MPI_COMM_WORLD);
```

```
MPI_Recv(in, N, MPI_DOUBLE, other, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Cautions about Deadlock

- MPI_Send returns when the send buffer is safe to reuse (not on delivery)
 - Small messages are usually buffered by the library (eager protocol), so the send returns right away
 - Large messages usually wait for the matching receive (rendezvous protocol)
- MPI_Recv holds until the copy is complete, so it too can hang

```
MPI_Send(out, N, MPI_DOUBLE, other, 0, MPI_COMM_WORLD);  
MPI_Recv(in, N, MPI_DOUBLE, other, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```
MPI_Sendrecv(out, N, MPI_DOUBLE, other, 0,  
             in, N, MPI_DOUBLE, other, 0,  
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Non-Blocking Comms

- MPI_Isend and MPI_Irecv start the operation and return a request handle right away
- MPI_Wait (single return) and MPI_Waitall (multiple returns) block until it completes
- MPI_Test checks and completes it only if it's done

```
MPI_Request reqs[2];  
MPI_Irecv(in, N, MPI_DOUBLE, other, 0, MPI_COMM_WORLD, &reqs[0]);  
MPI_Isend(out, N, MPI_DOUBLE, other, 0, MPI_COMM_WORLD, &reqs[1]);  
  
do_local_work();    /* must not use in or out */  
  
MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);
```

Making the Most: Collectives

- Every rank in the communicator is involved
- MPI_Barrier:
 - Wait until everyone arrives
- MPI_Bcast:
 - Root sends the same data to everyone
- MPI_Scatter and MPI_Gather:
 - Split data out from the root, collect it back
- MPI_Reduce and MPI_Allreduce:
 - Combine values with an operation such as MPI_SUM or MPI_MAX
- MPI_Allgather and MPI_Alltoall:
 - All data is sent and all data is received

Example: Global Sum

- Each rank computes a partial sum locally
- MPI_Reduce combines the partial sums at rank 0
 - Use MPI_Allreduce instead if every rank needs the result

```
double N[5000000] = ...
double local = 0.0, total = 0.0;
for (long i = rank; i < N; i += size)
    local += f(i);

MPI_Reduce(&local, &total, 1, MPI_DOUBLE, MPI_SUM,
          0, MPI_COMM_WORLD);

if (rank == 0)
    printf("total = %f\n", total);
```

Common Pitfalls

Errors:

- A receive buffer smaller than the incoming message will error
- Datatypes that don't match between sender and receiver might error
- Reusing a buffer before a nonblocking operation completes can corrupt data
- Relying on eager buffering (works for small messages) might fail
- Printout -- which rank are you on?
- Calling MPI functions before MPI_Init or after MPI_Finalize won't work

Performance:

- Remember every rank has to call a collective
- Printout from many ranks can overwhelm stdout

How to Write an MPI Program

1. Start small: 2 ranks and a subset of your data
2. Block out the work into overlapping chunks
3. Determine if you need blocking or non-blocking comms
4. Double-check outputs against a known-correct implementation or checkpoint
5. To begin to scale, check both even and odd rank counts

Example: Squaring (Send and Recv)

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char **argv) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        /* hand each worker a number */
        for (int dest = 1; dest < size; dest++) {
            int n = dest * 10;
            MPI_Send(&n, 1, MPI_INT, dest, 0, MPI_COMM_WORLD);
        }
        /* collect the answers, in rank order */
        for (int src = 1; src < size; src++) {
            int result;
            MPI_Recv(&result, 1, MPI_INT, src, 0, MPI_COMM_WORLD,
                    MPI_STATUS_IGNORE);
            printf("rank %d sent back %d\n", src, result);
        }
    }
    else {
        int n;
        MPI_Recv(&n, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        n = n * n;
        MPI_Send(&n, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
    }

    MPI_Finalize();
    return 0;
}
```

Example: Non-Blocking Ring

```
int main(int argc, char **argv) {
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    int right = (rank + 1) % size;
    int left  = (rank - 1 + size) % size;
    int out = rank * 100;
    int in  = -1;
    MPI_Request reqs[2];

    /* start both operations; neither call waits */
    MPI_Irecv(&in, 1, MPI_INT, left, 0, MPI_COMM_WORLD, &reqs[0]);
    MPI_Isend(&out, 1, MPI_INT, right, 0, MPI_COMM_WORLD, &reqs[1]);

    /* do some work that doesn't touch in or out */
    long local = 0;
    for (long i = 0; i < 1000000; i++)
        local += i % 7;

    /* after this, it's safe to reuse in, and to change out */
    MPI_Waitall(2, reqs, MPI_STATUSES_IGNORE);

    printf("rank %d got %d from rank %d (local work = %ld)\n", rank, in, left, local);

    MPI_Finalize();
    return 0;
}
```

Example: Timing

```
int main(int argc, char **argv) {
    int rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    /* higher ranks get more work */
    long iters = (rank + 1) * 20000000L;

    MPI_Barrier(MPI_COMM_WORLD);      /* start everyone together */
    double t0 = MPI_Wtime();

    double x = 0.0;
    for (long i = 0; i < iters; i++)
        x += 1.0 / (i + 1);

    double elapsed = MPI_Wtime() - t0;
    printf("rank %d: %.3f s (x = %f)\n", rank, elapsed, x);

    double fastest, slowest;
    MPI_Reduce(&elapsed, &fastest, 1, MPI_DOUBLE, MPI_MIN, 0, MPI_COMM_WORLD);
    MPI_Reduce(&elapsed, &slowest, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

    if (rank == 0)
        printf("fastest %.3f s, slowest %.3f s, timer resolution %g s\n",
            fastest, slowest, MPI_Wtick());

    MPI_Finalize();
    return 0;
}
```

Example: Wait and Status Use

```
int main(int argc, char **argv) {
    int rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == 1) {
        int data[3] = {4, 5, 6};
        MPI_Send(data, 3, MPI_INT, 0, 42, MPI_COMM_WORLD);
    }
    else if (rank == 0) {
        int buf[10];
        int count;
        MPI_Request req;
        MPI_Status status;

        /* post the receive, accept any tag */
        MPI_Irecv(buf, 10, MPI_INT, 1, MPI_ANY_TAG, MPI_COMM_WORLD, &req);
        printf("rank 0: receive posted, doing other work\n");

        /* block here until the message is in buf */
        MPI_Wait(&req, &status);

        MPI_Get_count(&status, MPI_INT, &count);
        printf("rank 0: got %d ints with tag %d and data: %d %d %d\n",
              count, status.MPI_TAG, buf[0], buf[1], buf[2]);
    }

    MPI_Finalize();
    return 0;
}
```

Up Next

- Syllabus correction coming out (with updated due dates)
- Project 2 (programming component) out tonight
 - Due in 3 weeks
- MPI and OpenMP Bonus Practice also out tonight
 - Due before the midterm